DA2304: Introduction to Computer Systems

Assignment 2 — Virtual Machines

Venkata Sreekar

April 2026

Part 1

Exercise 1 — Building the Hack VM Translator

The Python VM Translator (`HackVM.py` and `main.py`) has been implemented and is included in the `src/` folder of the submission ZIP archive.

Part 2

Exercise 1 — The MAC Operation Architecture

1.1. Sanity Checking

To safely evaluate $Y = WX + B$, the following two dimension constraints must hold:

1. The number of columns in W must equal the number of rows in X (required for the matrix product $W \cdot X$ to be defined).
2. The number of rows in W must equal the number of rows in B (required for the element-wise addition of WX and B).
3. The number of columns in X must equal the number of columns in B (required for the element-wise addition of WX and B).

Pseudocode — Dimension Validation:

```
function SanityCheck(W.rows, W.cols, X.rows, X.cols, B.rows, B.cols):

   // 1. Check dot-product compatibility
if (W.cols ≠ X.rows): goto THROW_ERROR

   // 2. Check bias matrix compatibility
if (B.rows ≠ W.rows): goto THROW_ERROR
if (B.rows ≠ X.rows): goto THROW_ERROR

   // All checks passed — signal success
Memory[ERROR_FLAG_ADDR] ← 1
return

label THROW_ERROR:
// Signal failure
Memory[ERROR_FLAG_ADDR] ← −1
return
```

In the above pseudocode, ERROR_FLAG_ADDR can be taken as 255.

Exercise 2 — Architectural Analysis

1. Stack Usage

Approach A (MATMUL): The stack is used minimally because the entire operation executes inside a single function invocation. The caller's state is saved only once, before computation begins, which eliminates repeated push/pop overhead and results in fast execution.

Approach B (ROWXCOL): The stack is exercised intensively because the subroutine is invoked once per output matrix element. The runtime must save the caller state and pass fresh arguments on every call, which can amount to hundreds of redundant stack operations and significantly degrades performance.

2. Segment Usage

Approach A (MATMUL): This approach relies heavily on the `local` segment to maintain loop counters and temporary variables. The `pointer` and `that` segments are used directly for matrix memory access, while the `argument` segment is consulted only at initialisation.

Approach B (ROWXCOL): The `argument` segment is used constantly, receiving a fresh row pointer, column pointer, and length on every subroutine call. The `local` segment carries minimal state per call (only the running sum and loop counter). The `pointer` and `that` segments remain necessary for dereferencing matrix addresses.

3. Optimization — Approach A

In software, the inner loops can be accelerated by computing the next element's address via pointer increment (adding a fixed stride) rather than re-deriving it through full index multiplication on each iteration. At the hardware level, integrating a dedicated *Multiply-Accumulate* (MAC) unit directly into the CPU datapath would allow the innermost arithmetic to execute in a single clock cycle.

4. Optimization — Approach B

The principal bottleneck of Approach B is the per-element subroutine call overhead. A compiler-level technique called *Function Inlining* eliminates this by substituting the body of RowXCol directly at each call site, so the programmer retains readable, modular source code while the generated assembly incurs none of the stack-management cost at runtime.

Extra Credit

My recent favourite song is *gallo telinattunde* from the film *Jalsa* (2024).